19: Lookup Table Approximation

Engr 315: Hardware / Software Codesign

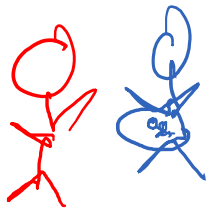
Andrew Lukefahr
Indiana University

Some material taken from EECS370 at U. of Michigan

The Zoom / Canvas Lecture Plan

- Lecture using Zoom:
  - https://iu.zoom.us/j/362983973
  - Password: ENGR315

- Also available on Canvas
  - Next day -ish



Office Hours

- Ethan is hosting virtual office hours:

- Wed 5:30 PM - 8:00 PM, Th 2:30 PM - 4:00 PM
- https://iu.zoom.us/j/770949556

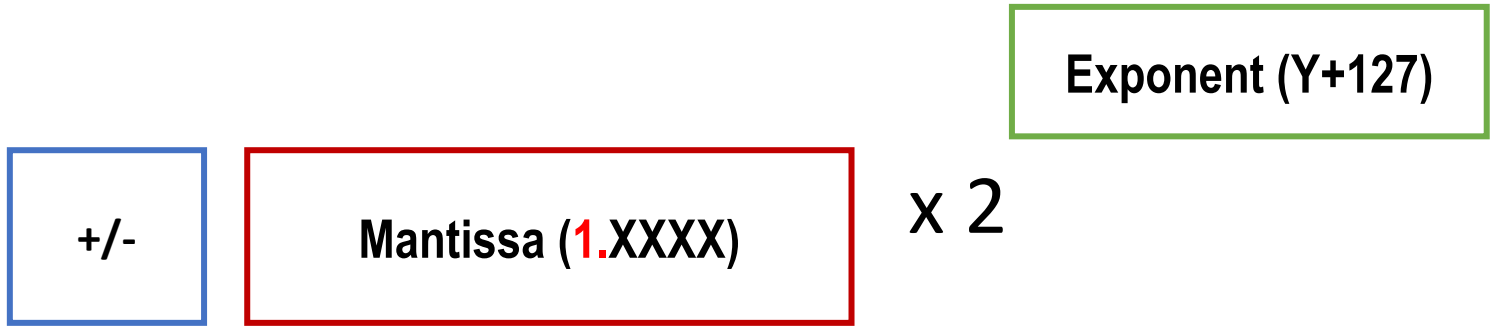
P4: Bitcounting

- Due on Friday
- Austin is going guest lecture
  - Present C implementation.
  - Build a simple Linux kernel module to help

PX: Neural Networks

- We're going to scale this back a little.
- Focus on Tanh ~~Sigmoid~~
- Beyond that is going to be extra credit.

Review: IEEE-754 “Float” Standard



Float Example

Exponent (Y+127)

+/- | Mantissa (1.XXXX) | x 2

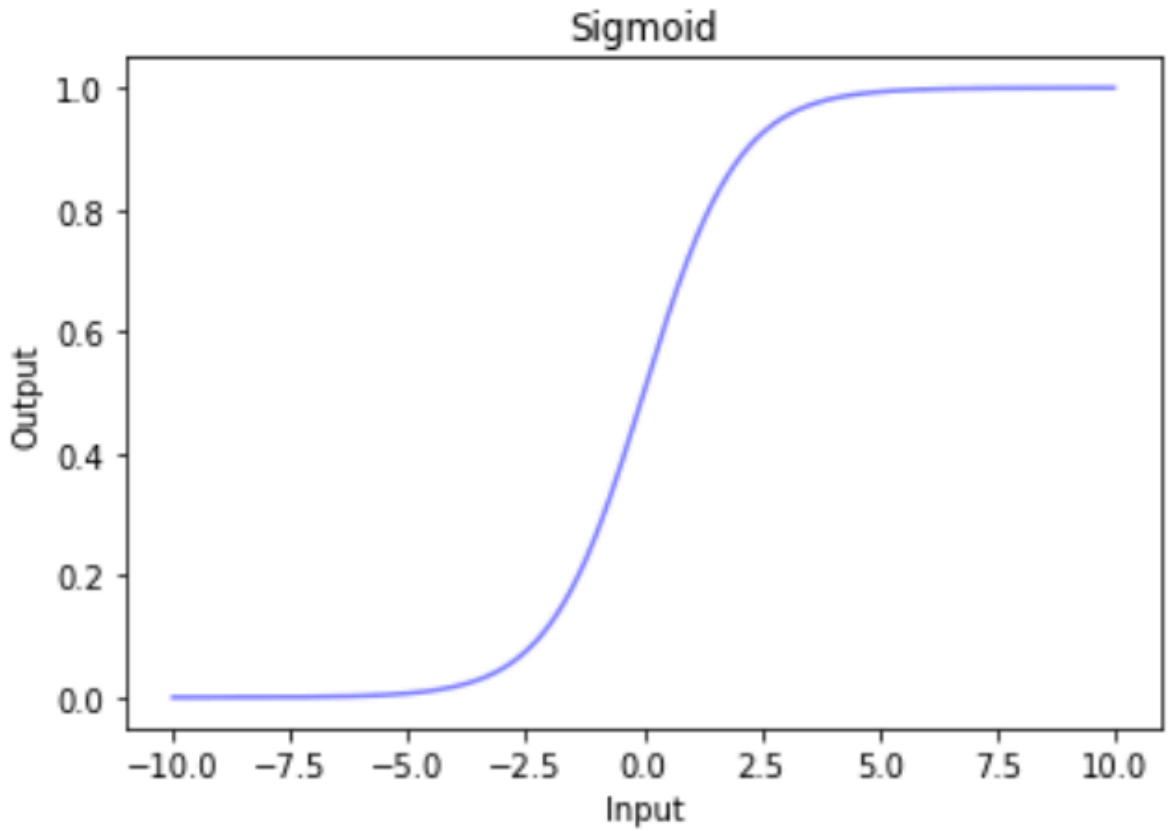
- $+10.625_{10}$ => $+1010.101_2$ => $+1.010101 \times 2^3$
- Sign: + => 0_2
- Mantissa: ~~1~~.**010101** => **0101 0100 0000 0000 0000 000$_2$**
- Exponent: $3_{10} + 127_{10} = 130_{10}$ => **1000 0010$_2$**

Mult/Div is "easy"er

Add/Sub are hard

10.625_{10} = 0 10000010 01010100000000000000000

Sigmoid is Floating-Point




```
def sigmoid(x):
    return 1 / (1 + np.e ** -x)
```

$$\frac{1}{1+e^{-x}}$$

Sigmoid Computation

floating point

1+~

e^{-x}

```
def sigmoid(x):
    return 1 / (1 + np.e ** -x)
```

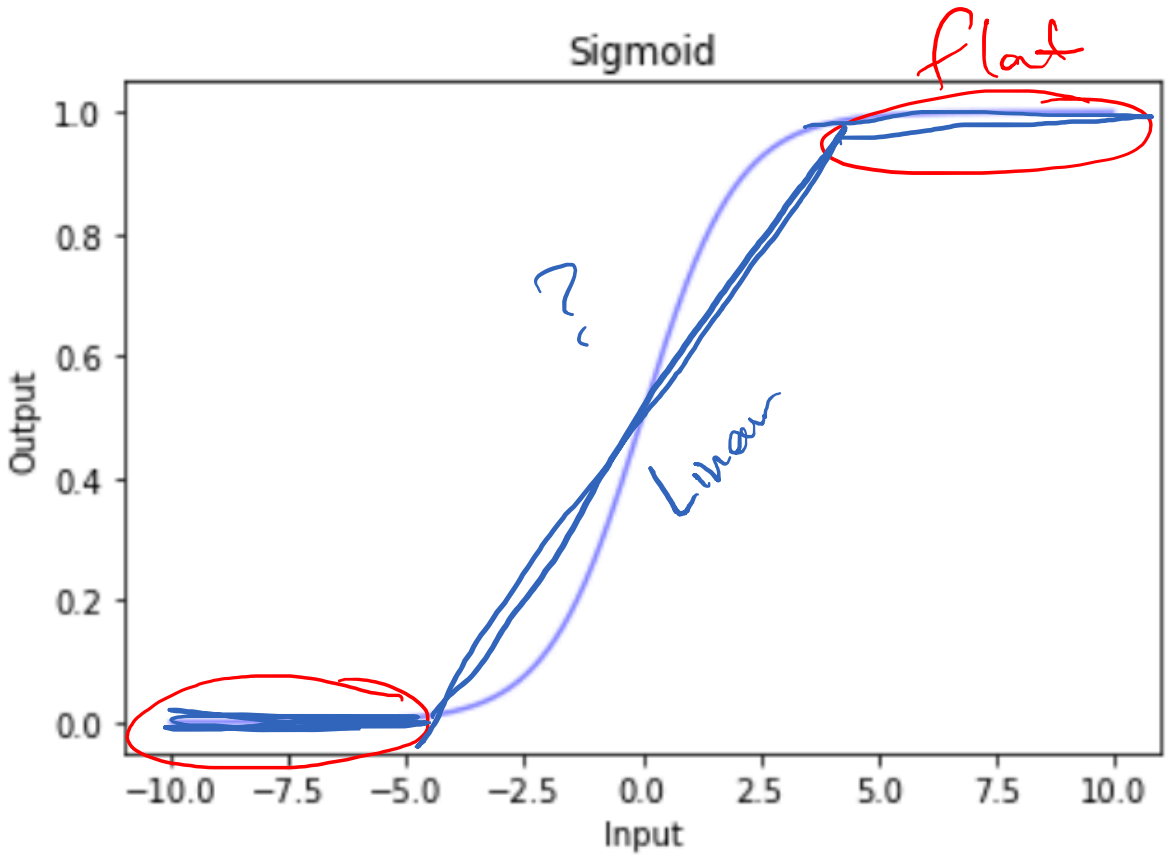
- 1 negation
- 1 floating-point exponential
- 1 floating-point addition
- 1 floating-point division

- Can we make this simpler?

Approximating Sigmoid

$x > 5 \Rightarrow y = 1$

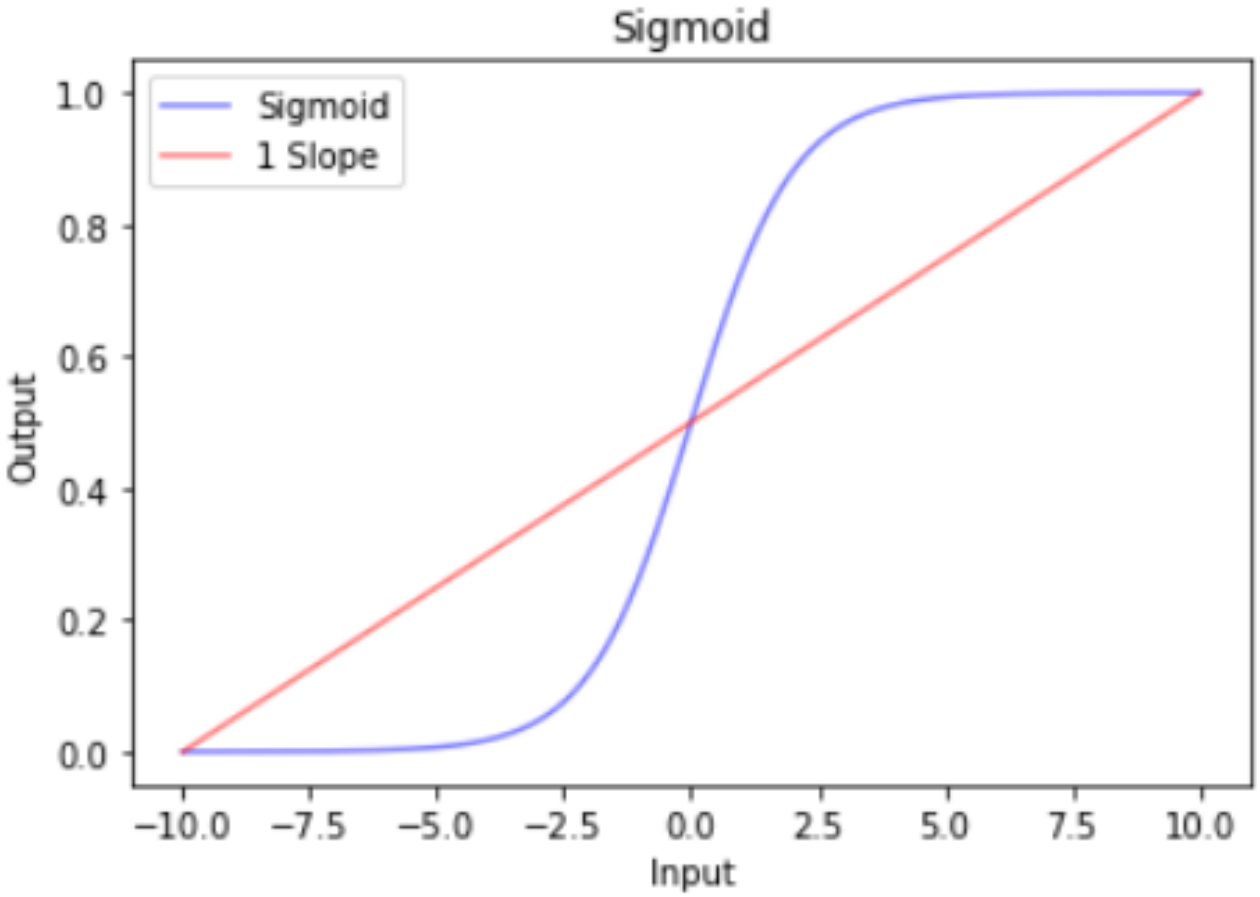
$\frac{1}{1+e^{-x}}$



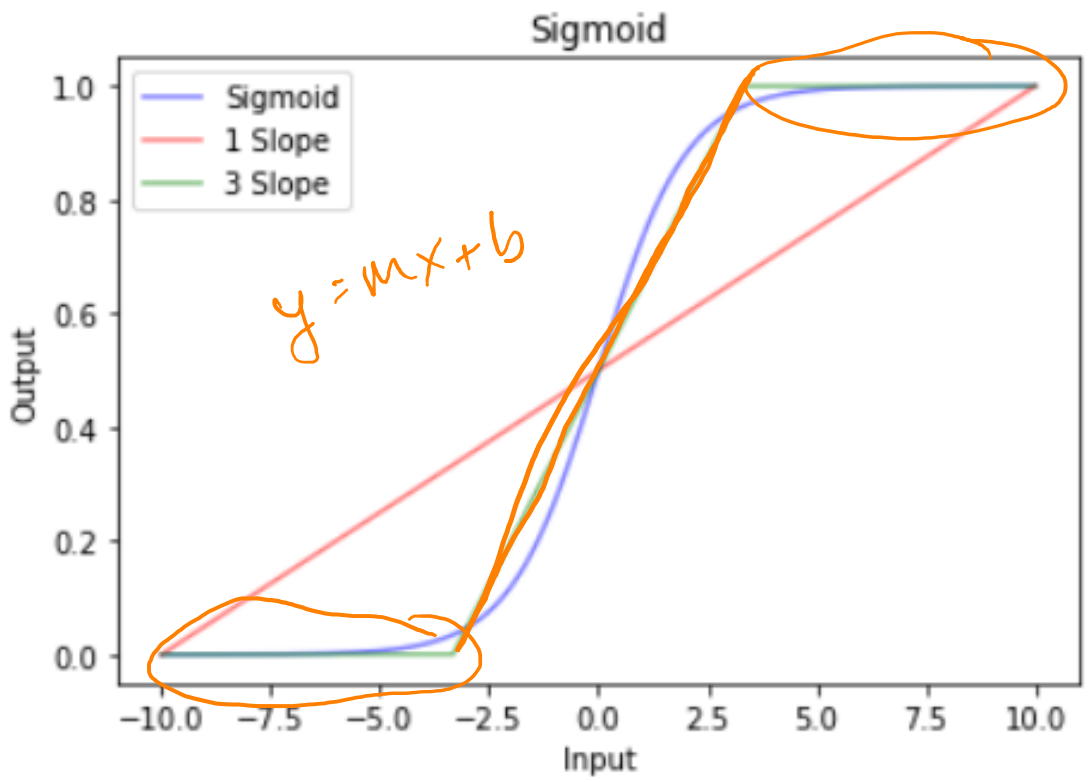
$x < -5$
$y = 0$

Ways to approximate Sigmoid?

Linear Approximation



Piece-wise Linear Approximation



1

0

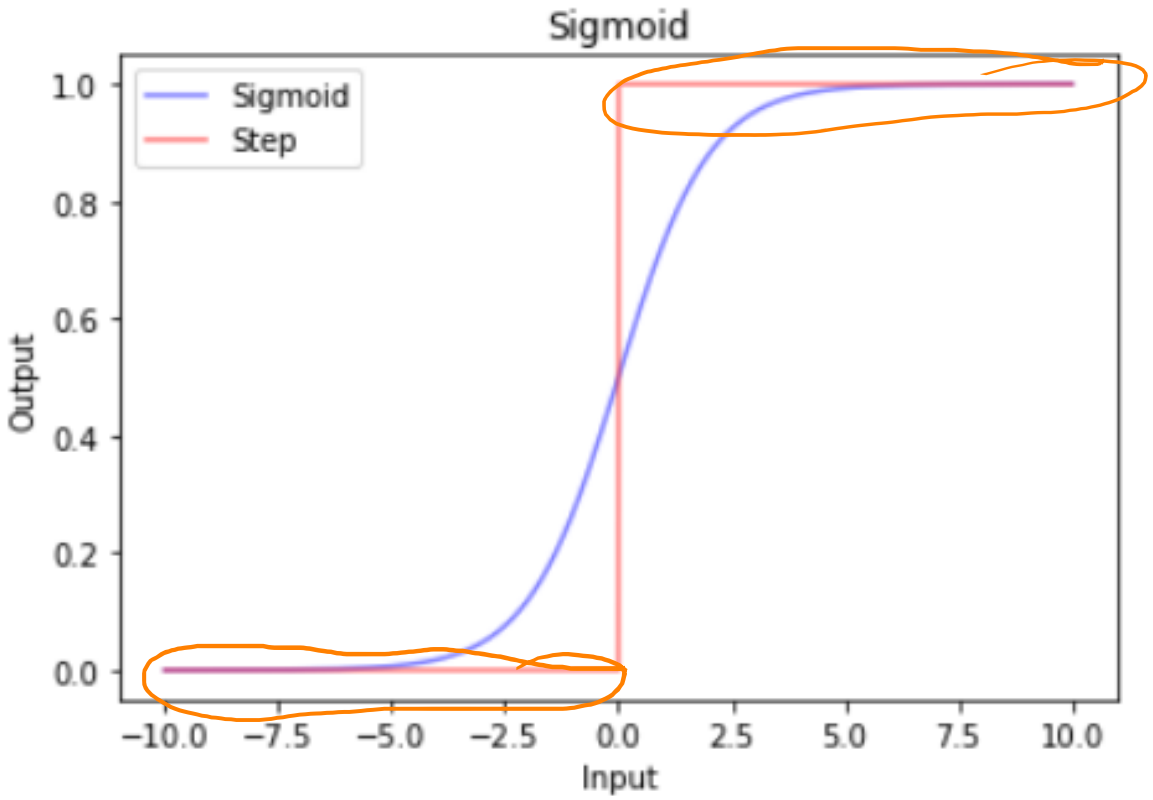
$y = mx + b$

Lines that have slopes

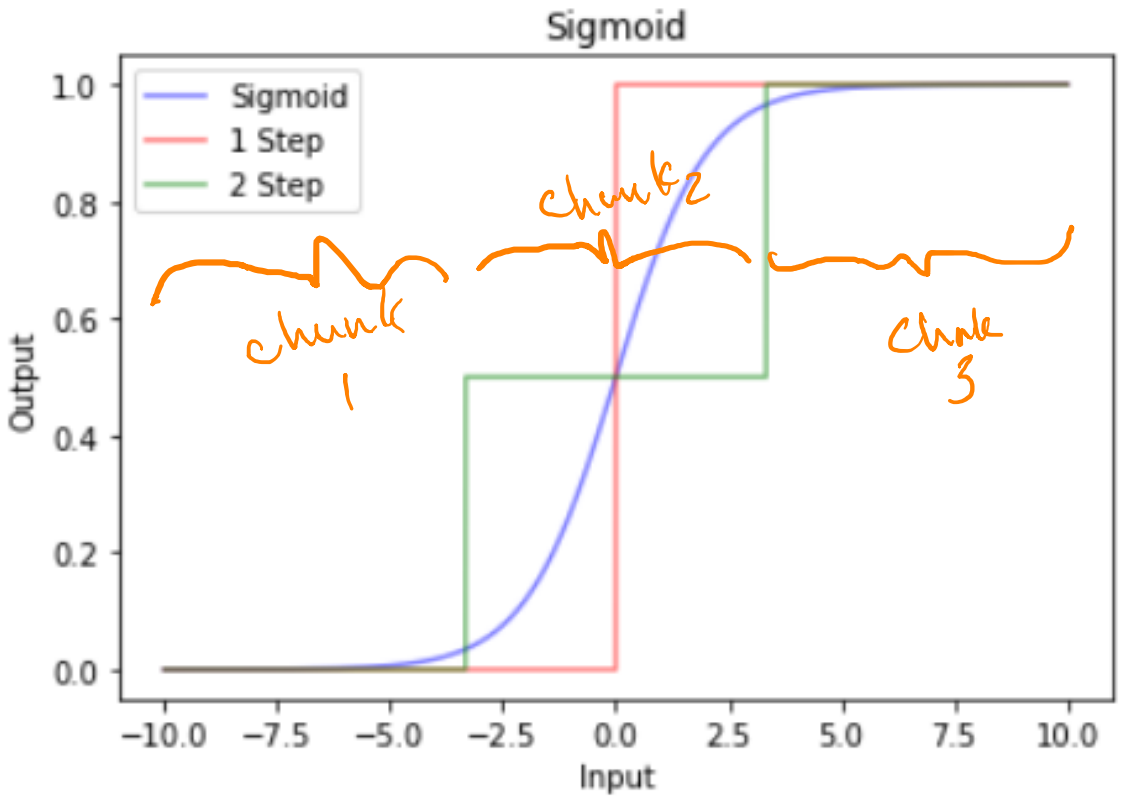
slope = m $x + b$

line = b

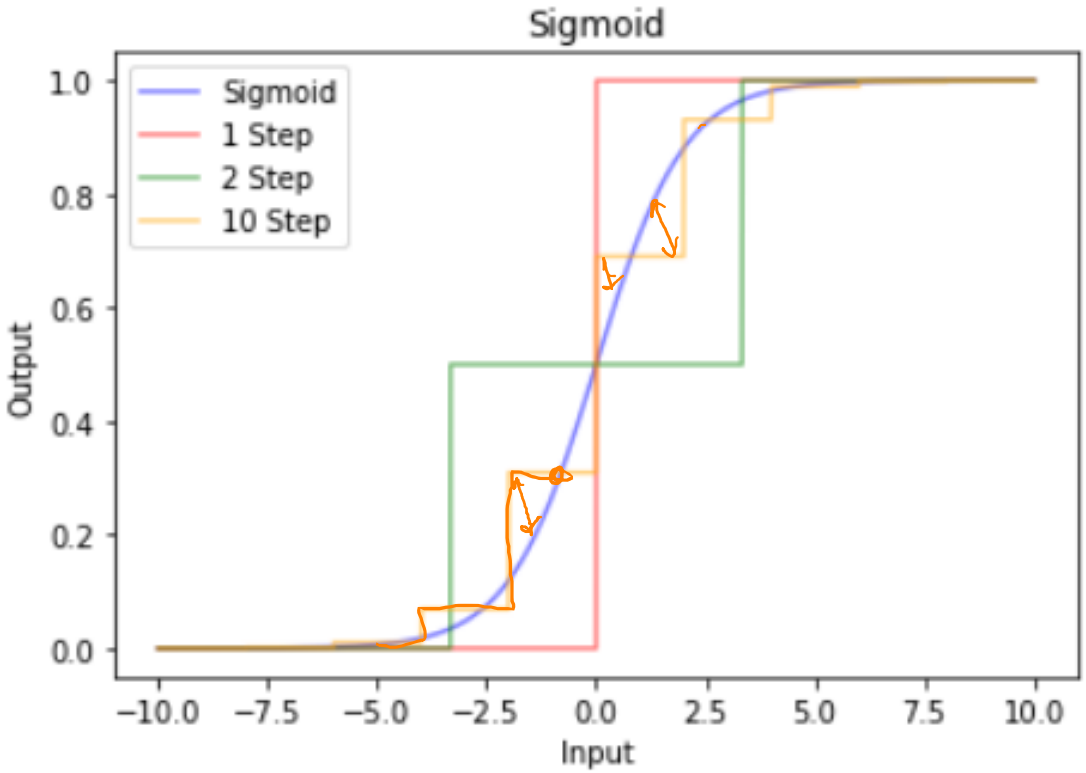
Step Approximation



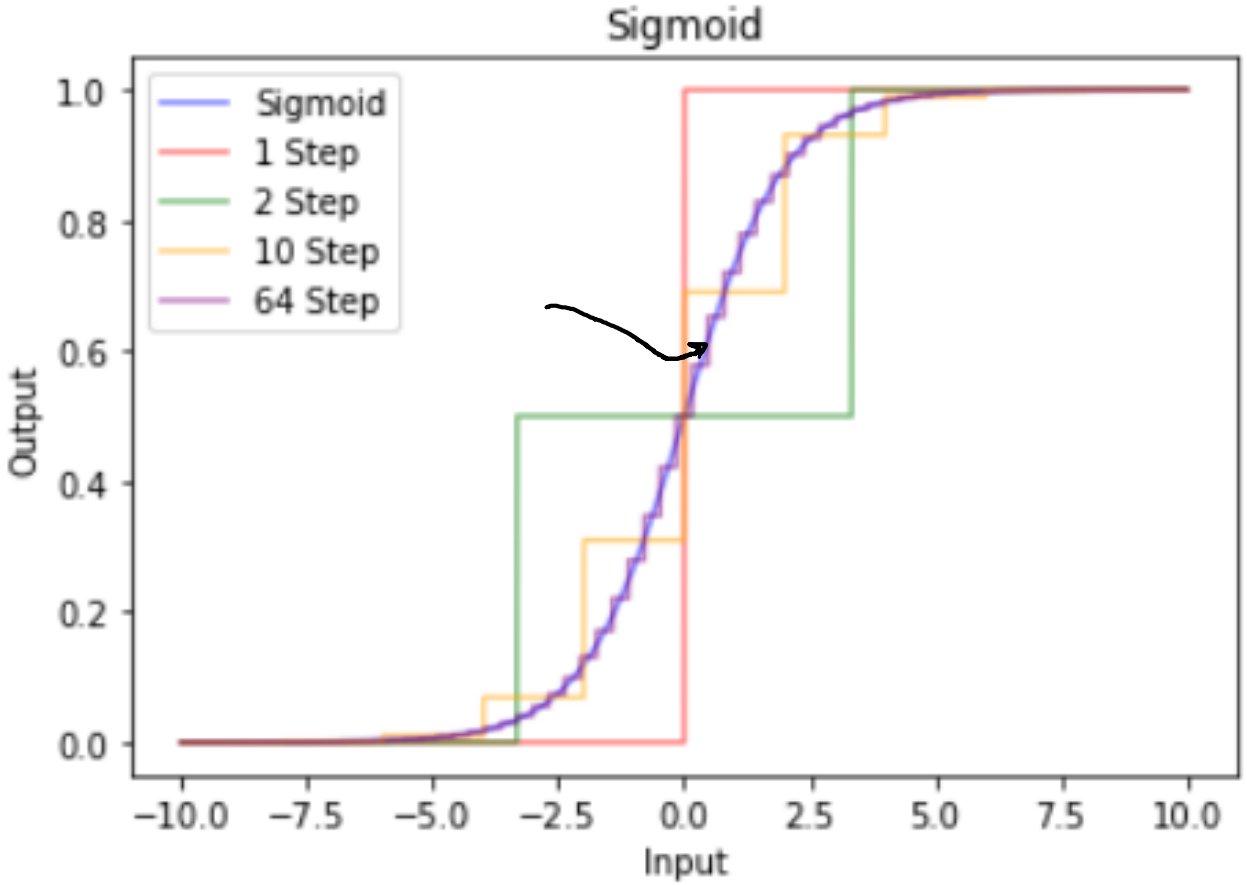
Multi-Step Approximation



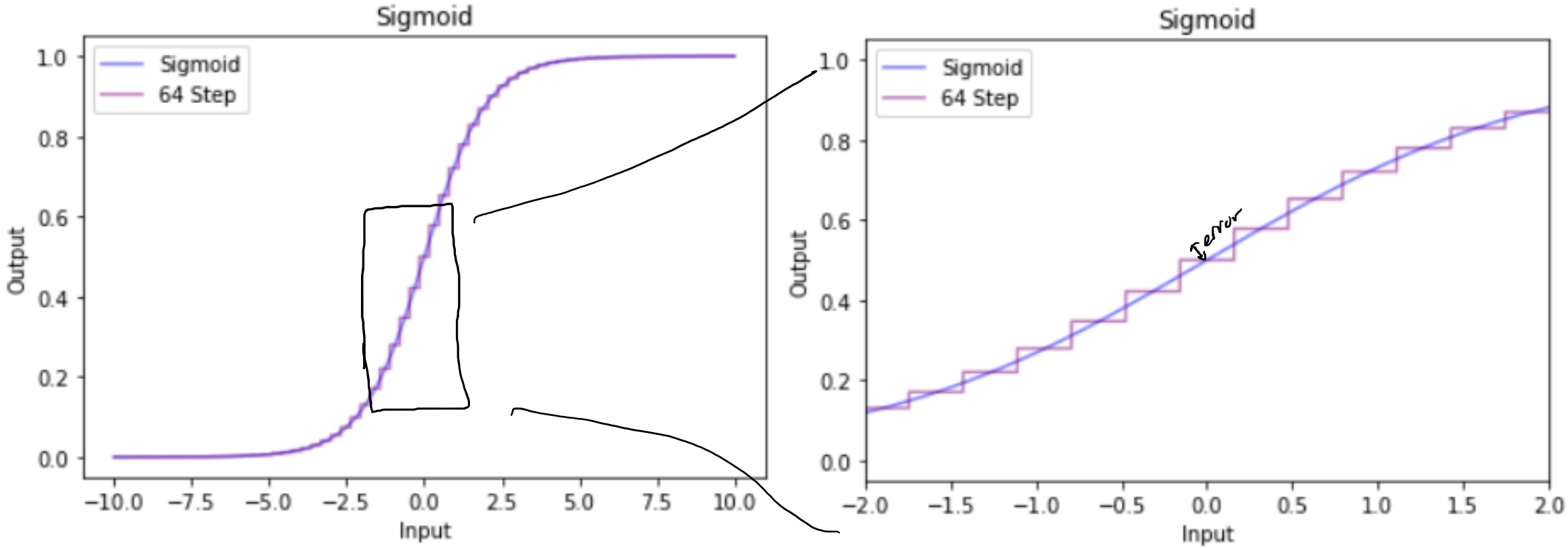
Many-Step Approximation



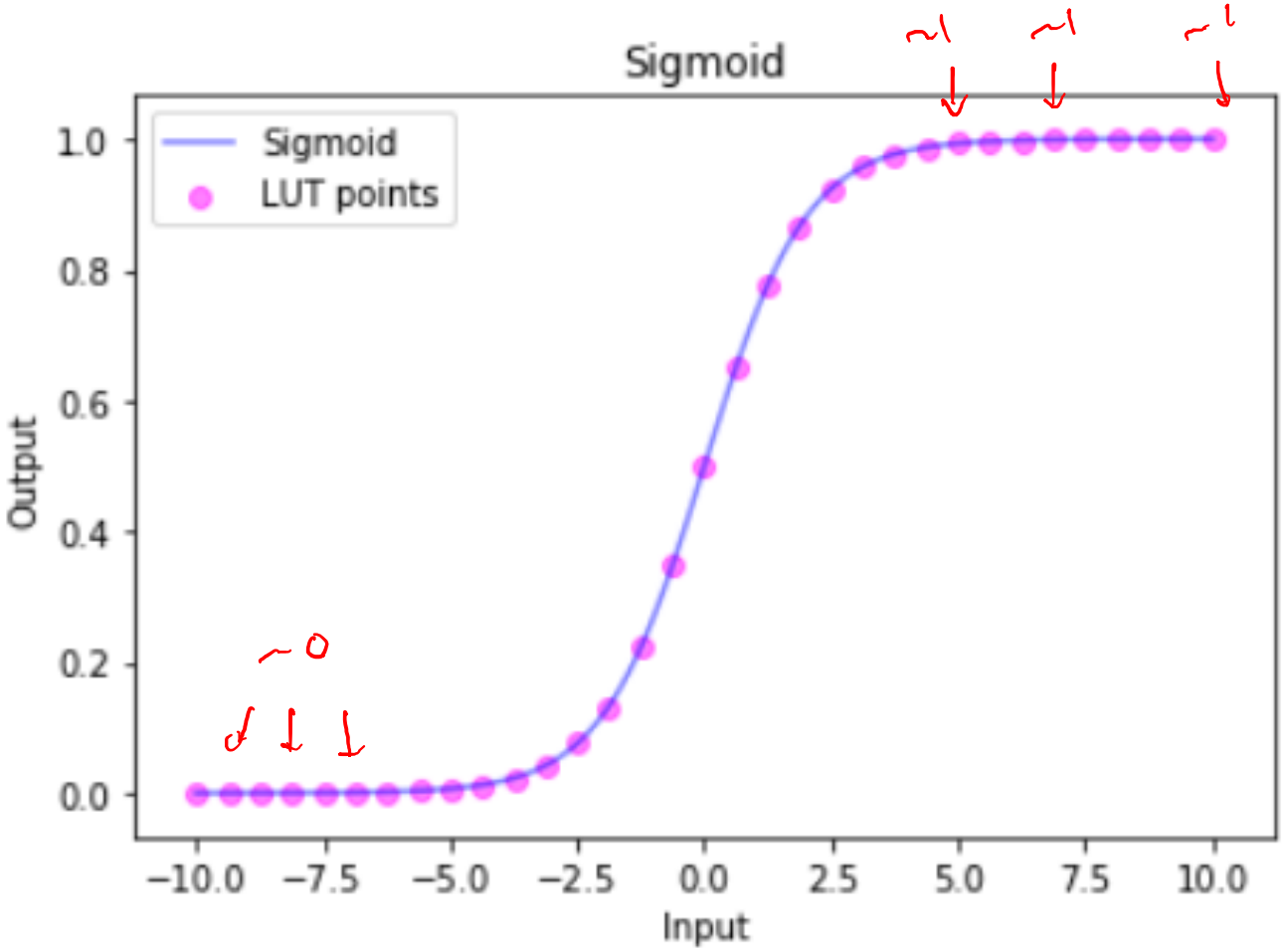
Many-Step Approximation



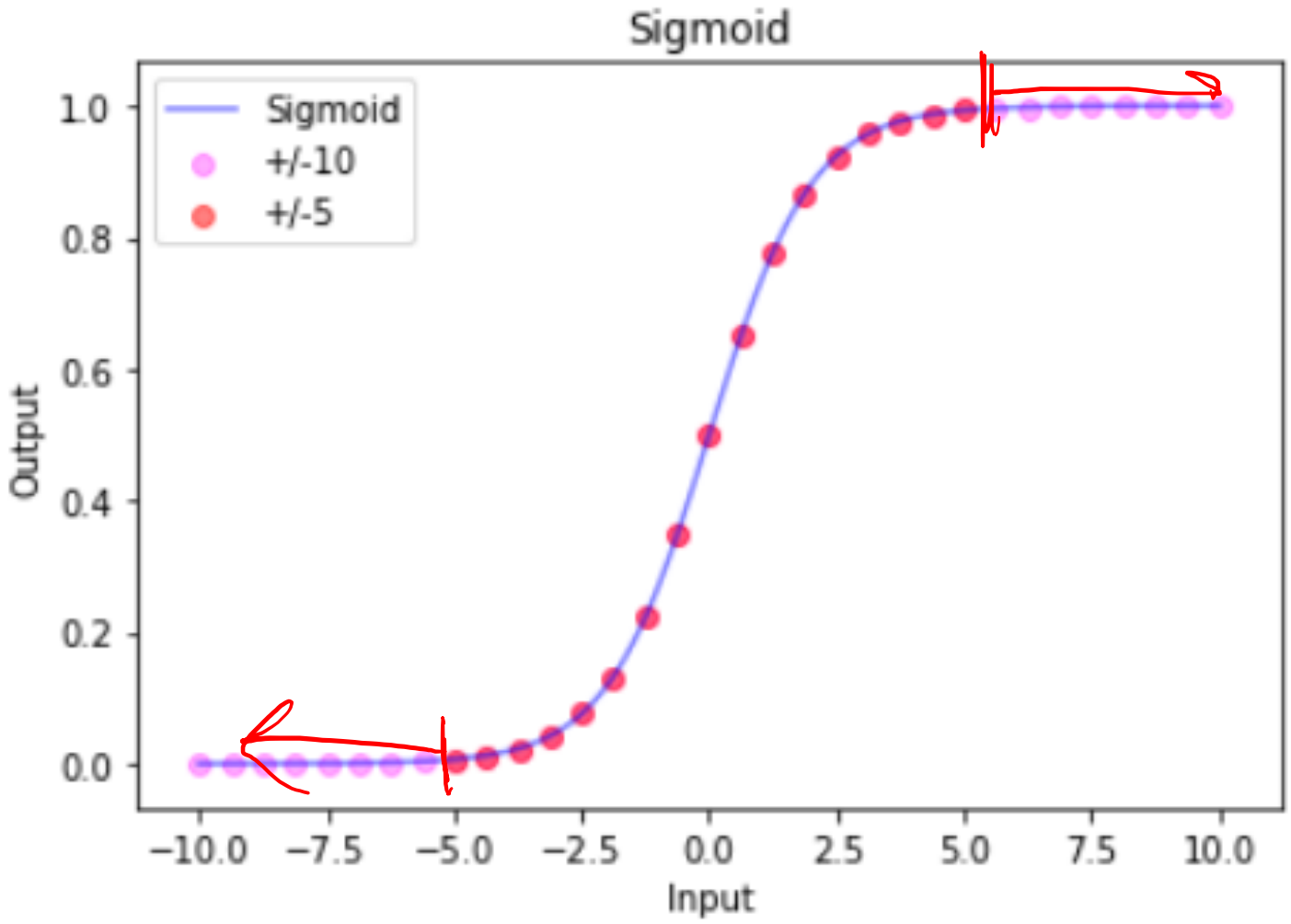
Approximated Sigmoid



Pre-Calculate Sigmoid points

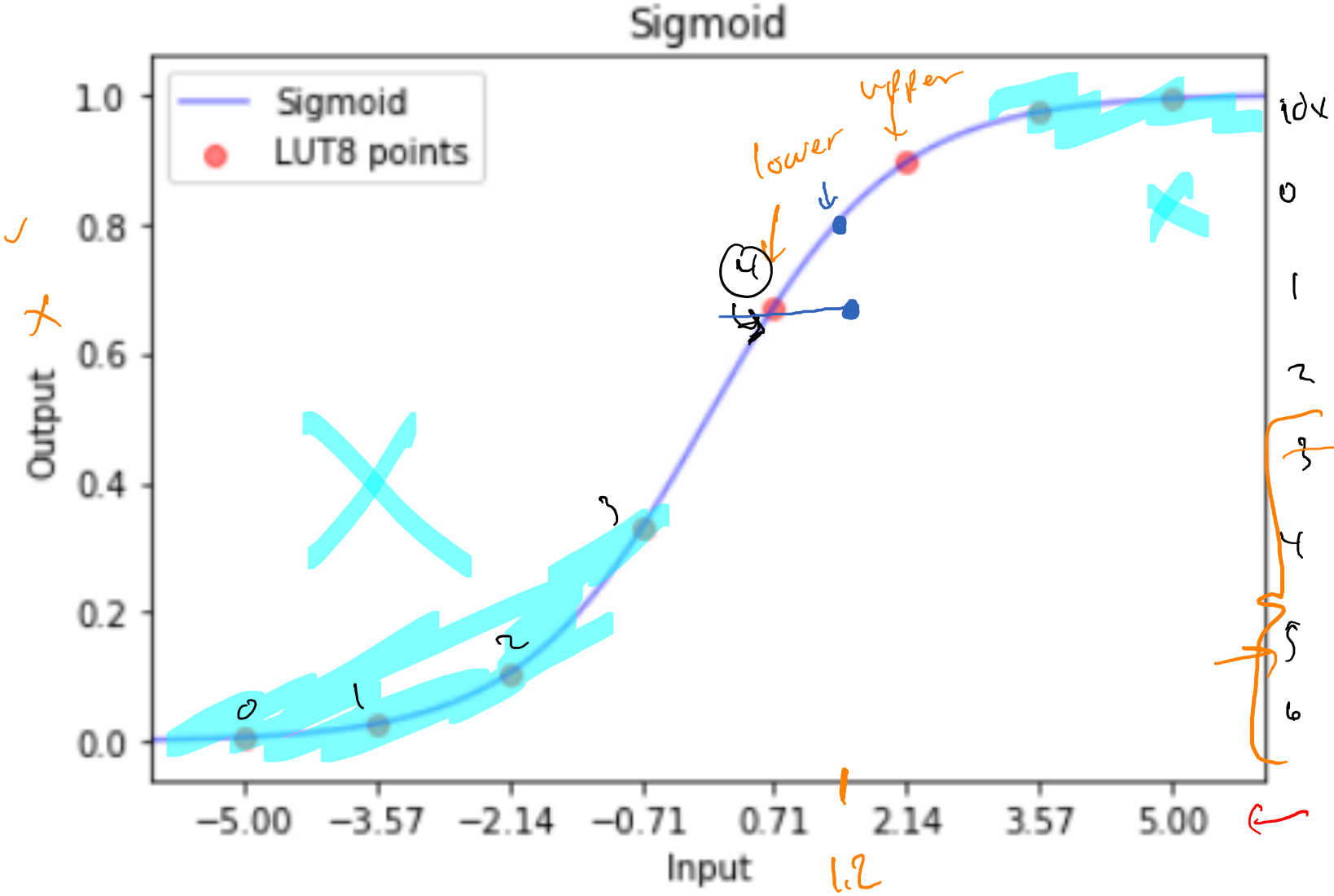


Optimization: Remove redundant values



How do we find the nearest point?

y = Sigmoid (x=1.2)

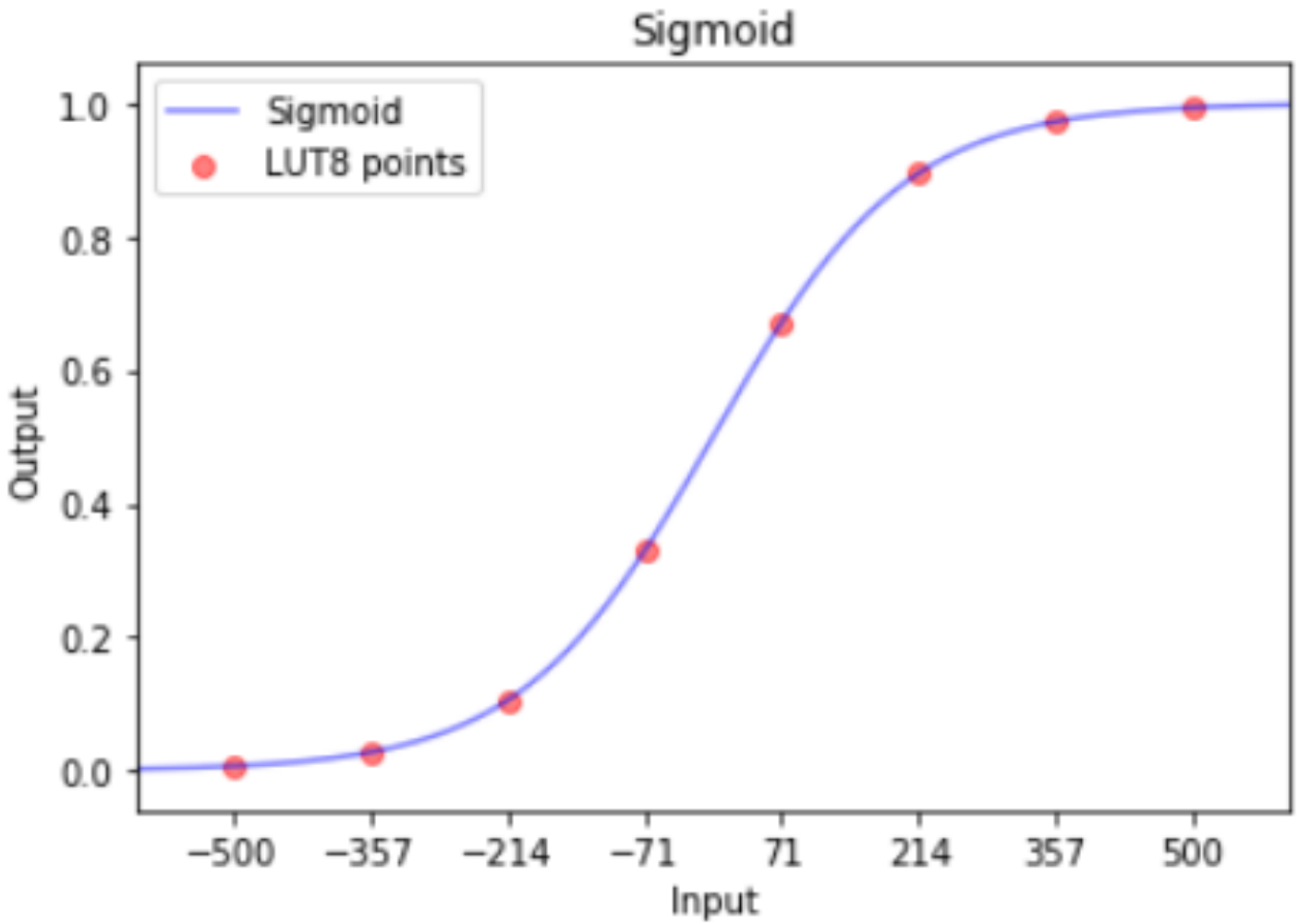


1.2 > -0.71 ✓

1.2 ? 3.57 ✗

idx	x	y
0	-5	0
1	-3.57	0.1
2	-2.14	0.2
3	~~-0.71~~	0.4
4	+0.71	0.6
5	+3.57	0.9
6	+5	1.0

What about now? (X-axis scaled)



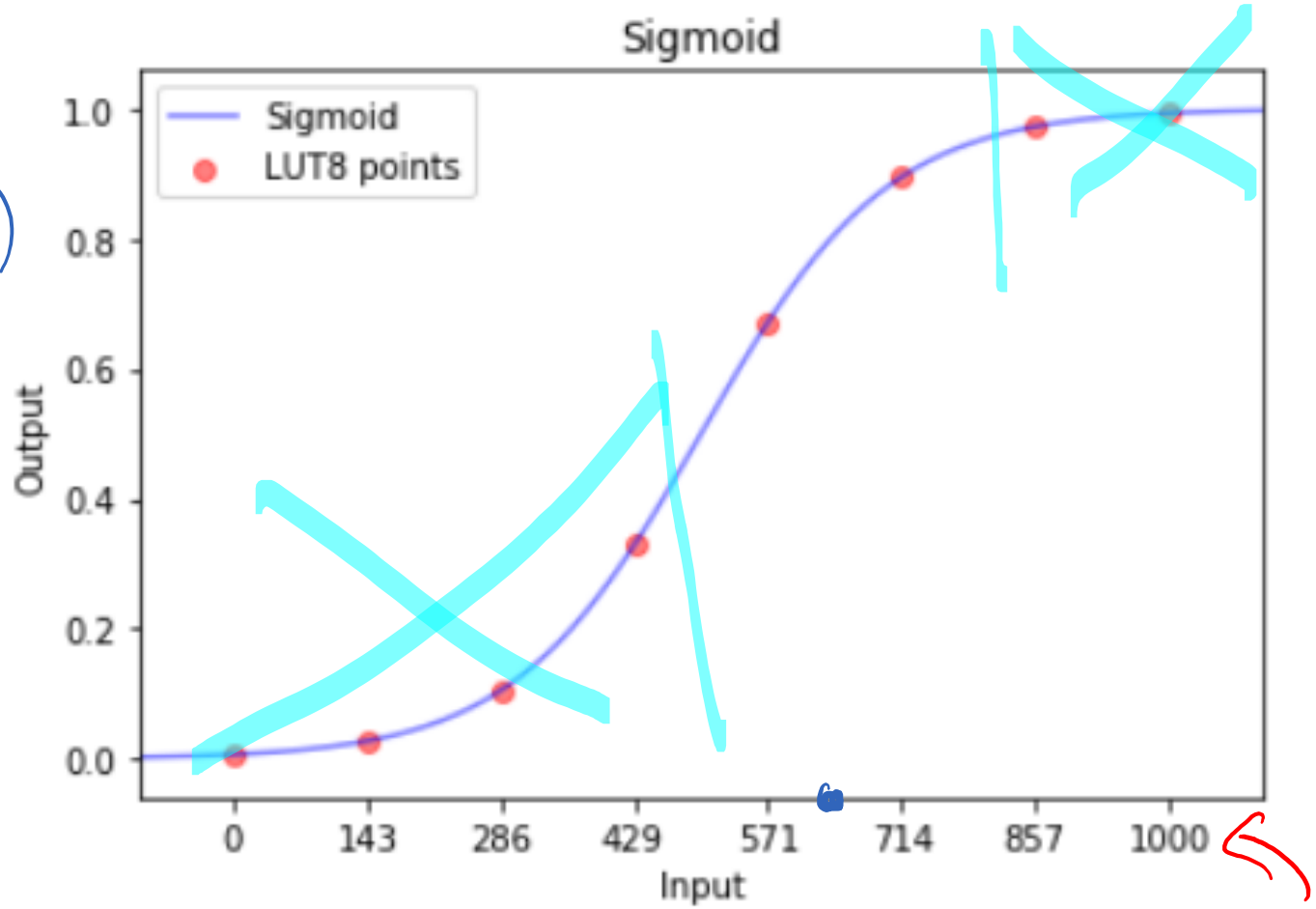
x 100

What about now? (X-axis shifted)

Sigmoid(x=1.2)

(1.2 x 100) +500

= 620



x	y
0	
143	
286	
429	
571	
714	
857	
1000	

x100 +500

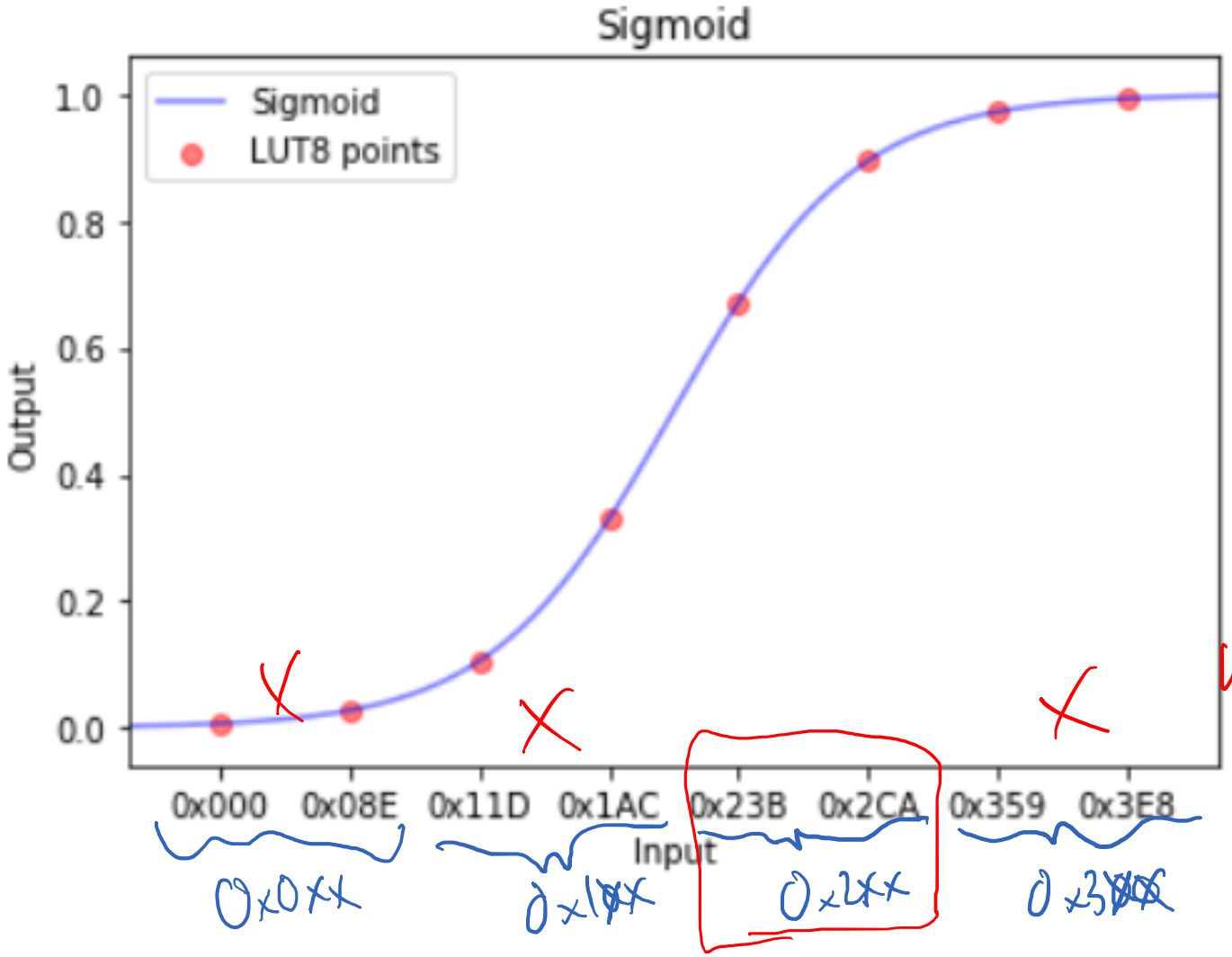
What about now? (X-axis radix)

sigmoid(1.2)

1.2 × 100 + 500 = 620

hex(620)

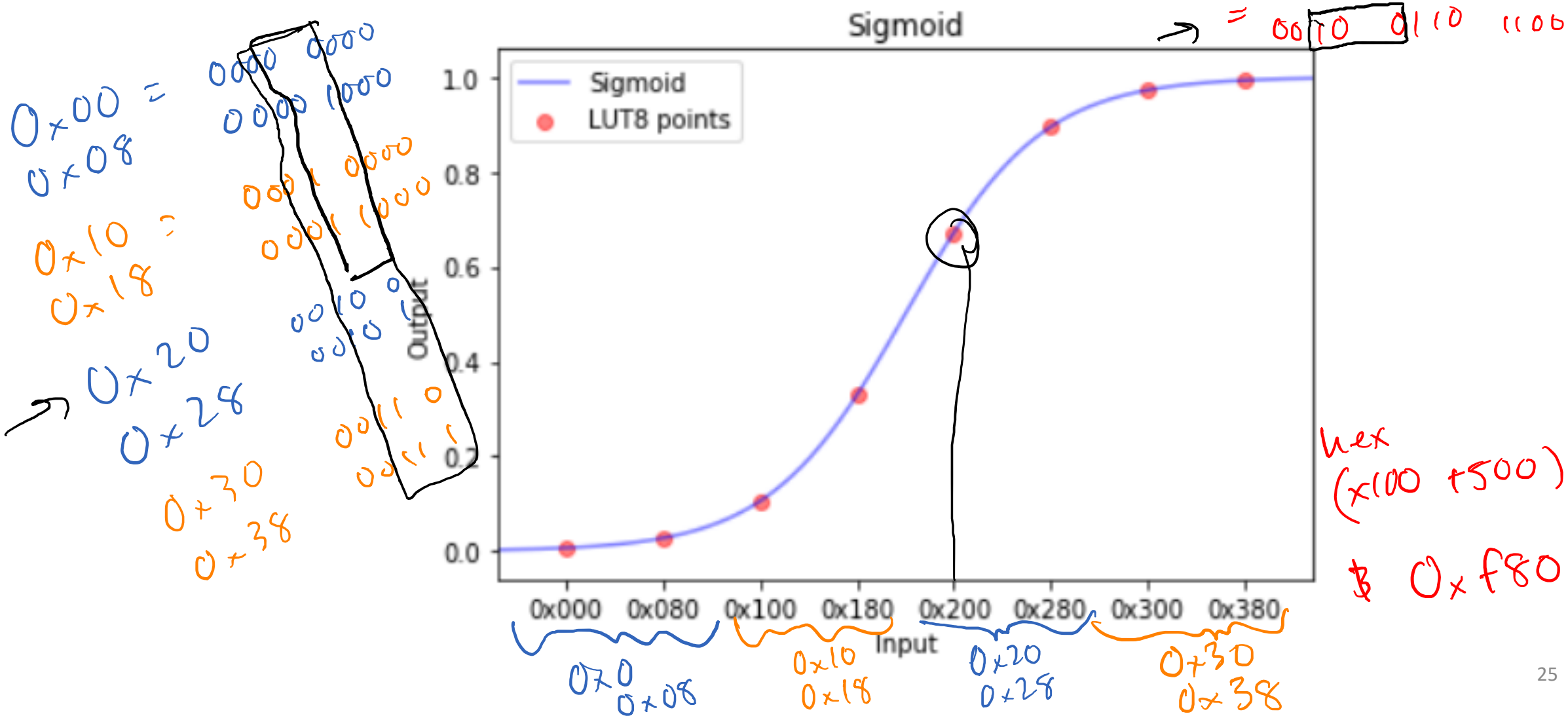
0x26C



hex(x100 + 500)

What about now? (X-axis masked)

Sigmoid (1.2)
(×100 +500)
= 620_{10} = $26C_2$
= 0010 0110 1100



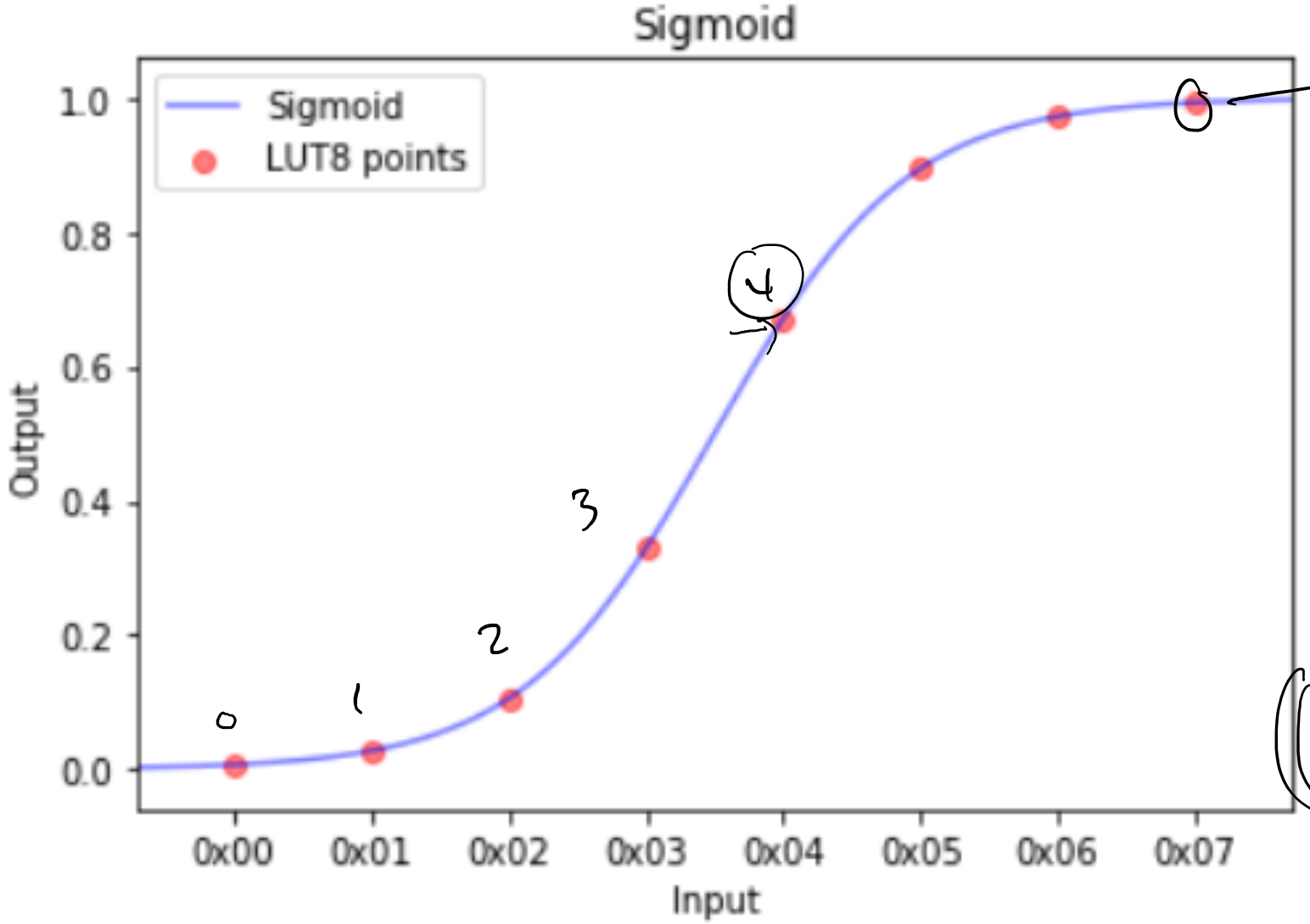
What about now? (X-axis bitshifted)

Sigmoid(x = 1.2)

0x26c

0010 0110 1100

= idx = 4



(((x × 100) + 500) & 0xf80) >> 7

How do we generate an index?

array

Index	X	Y
0	-5.000	0.007
1	-3.571	0.027
2	-2.143	0.105
3	-0.714	0.329
4	0.714	0.671
5	2.143	0.895
6	3.571	0.973
7	5.000	0.993

0

10.0

$\rightarrow -5.0 \le x < +5.0$

$+5.0 \rightarrow 0 \le x+5 < 10$

$5-(-5)=10$

$\times 8$

sigmoid $(x = 4.999) = \lfloor 7.9992 \rfloor = 7$

floating point

```
def gen-idx(x):
    return int(((x + 5.0) / 10 · 8))
```

array[gen-idx(1.2)].y = 0.671

$0 \le \frac{x+5}{10} < 1.0$

$0 \le \frac{x+5}{10} \cdot 8 < 8.0$

int

$0 \le \text{int}\left(\frac{x+5}{10} \cdot 8\right) \le 7$

How do we generate an index?

Sigmoid Table

Index	X	Y
0	-5.000	0.007
1	-3.571	0.027
2	-2.143	0.105
3	-0.714	0.329
4	0.714	0.671
5	2.143	0.895
6	3.571	0.973
7	5.000	0.993

```
def gen_idx(x):
    return int((x + offset)/scale * arr_len)
```

+50 10.0 8

8/10

```
scale2 = arr_len / scale
def gen_idx2(x):
    return int((x+offset) * scale2)
```

$$\text{sigmoid} = \frac{1}{1+e^{-x}}$$

gen_idx2(1.2) => 4

sig_table[4] = 0.671

What about gen_idx2(-10)? Gen_idx2(+10)?

Index	X	Y
0	-5.000	0.007
1	-3.571	0.027
2	-2.143	0.105
3	-0.714	0.329
4	0.714	0.671
5	2.143	0.895
6	3.571	0.973
7	5.000	0.993

```
scale2 = arr_len / scale
def gen_idx2(x):
    return int((x+offset) * scale2)
```

$(-10+5)\cdot\frac{8}{10} = -4$

What about gen_idx2(-10)? Gen_idx2(+10)?

Index	X	Y
0	-5.000	0.007
1	-3.571	0.027
2	-2.143	0.105
3	-0.714	0.329
4	0.714	0.671
5	2.143	0.895
6	3.571	0.973
7	5.000	0.993

```
scale2 = arr_len / scale
def gen_idx2(x):
    return int((x+offset) * scale2)
```

```
def gen_idx3(x):
    if (x <= -5): return 0
    elif (x >= +5): return 7
    else: return int((x+offset) * scale2)
```

$(a+b)\cdot c = a\cdot c + b\cdot c$

FP Addition is expensive.

Index	X	Y
0	-5.000	0.007
1	-3.571	0.027
2	-2.143	0.105
3	-0.714	0.329
4	0.714	0.671
5	2.143	0.895
6	3.571	0.973
7	5.000	0.993

```
def gen_idx3(x):
    if (x <= -5): return 0
    elif (x >= +5): return 7
    else: return int((x+offset) * scale2)
```

FP Addition is expensive.

Index	X	Y
0	-5.000	0.007
1	-3.571	0.027
2	-2.143	0.105
3	-0.714	0.329
4	0.714	0.671
5	2.143	0.895
6	3.571	0.973
7	5.000	0.993

```python
def gen_idx3(x):
    if (x <= -5): return 0
    elif (x >= +5): return 7
    else: return int((x+offset) * scale2)
```

$$\text{int}\left((x \cdot \text{scale2})\right) + \overbrace{(\text{offset} \cdot \text{scale2})}^{\text{constant}}$$

FP Addition is expensive.

Index	X	Y
0	-5.000	0.007
1	-3.571	0.027
2	-2.143	0.105
3	-0.714	0.329
4	0.714	0.671
5	2.143	0.895
6	3.571	0.973
7	5.000	0.993

```
def gen_idx3(x):
    if (x <= -5): return 0
    elif (x >= +5): return 7
    else: return int((x+offset) * scale2)
```

```
scale3 = int(scale2 * offset)
def gen_idx4(x):
    if (x <= -5): return 0
    elif (x >= +5): return 7
    else: return int(x*scale2) + scale3
```

```
def gen_idx5(x):
    idx = int(x*scale2) + scale3
    if (idx < 0): return 0
    if (idx > 7): return 7
    else: return idx
```

fp

(int)

int

int

PX: Current Plan

$$\tanh = \frac{e^{+x} - e^{-x}}{e^{+x} + e^{-x}}$$

- Shift to using tanh (not sigmoid)
- You get to build a LUT for tanh
- Still waiting to see if that will be faster. ☺

Float to Hex In Python

32-int
int

32 bit

```
import struct
def float_to_hex(f):
    return hex(struct.unpack('<I', struct.pack('<f', f))[0])
```

string

```
flts = [-16, -2.1, -1.2, 0, 1.2, 5.0, 32.999, 42.0]
flts_hex = list(map(float_to_hex, flts))
print (list(zip(flts, flts_hex)))
```

```
[(-16, '0xc1800000'), (-2.1, '0xc0066666'), (-1.2, '0xbf9999
9a'), (0, '0x0'), (1.2, '0x3f99999a'), (5.0, '0x40a00000'),
(32.999, '0x4203fefa'), (42.0, '0x42280000')]
```

19: Lookup Table Approximation

Engr 315: Hardware / Software Codesign

Andrew Lukefahr
Indiana University